

NovaFreight (Demo Company)



Web Application Penetration Test Report

Assessment Type: Black Box Penetration Test

Engagement Dates: September 1 – September 3, 2026

Report Date: September 3, 2026

Prepared By: Ammar Ehab, Lead Penetration Tester — TCM Security

Prepared For: NovaFreight — John Smith, Global Information Security Manager

CONFIDENTIAL

Table of Contents

Confidentiality Statement 4

Disclaimer..... 4

Contact Information 4

Assessment Overview 5

Assessment Components 5

 External Penetration Test..... 5

Finding Severity Ratings 6

Risk Factors..... 6

 Likelihood..... 6

 Impact 7

Scope..... 7

 Scope Exclusions..... 7

Executive Summary 7

 Scoping and Time Limitations..... 7

 Testing Summary..... 8

 Tester Notes and Recommendations 8

Detailed Findings 11

Finding: SQL Injection (Authentication Bypass) (Critical)..... 11

Finding: SQL Injection (Data Extraction) (Critical) 12

Finding: Broken Access Control (Admin Panel) (Critical)..... 13

Finding: Unrestricted File Upload (Critical)..... 16

Finding: Stored Cross-Site Scripting (XSS) (High)18

Finding: Path Traversal (High)19

Finding: Weak Credential Storage (High)20

Finding: Cross-Site Request Forgery (CSRF) (High)21

Vulnerability Summary & Report Card 23

 Internal Penetration Test Findings23

Remediation Roadmap: Short-, Medium-, and Long-Term Priorities..... 9

 Short-Term (0-30 Days)9

 Medium-Term (1-3 Months).....9

 Long-Term (3-6+ Months, Ongoing Program)10

Appendix A: Tester Scope Notes..... 25

Confidentiality Statement

This document is the exclusive property of NovaFreight and Ammar. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires consent of both NovaFreight and Ammar.

Demo Corp may share this document with auditors under non-disclosure agreements to demonstrate penetration test requirement compliance.

Disclaimer

A penetration test is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment and not any changes or modifications made outside of that period.

Time-limited engagements do not allow for a full evaluation of all security controls. Ammar prioritized the assessment to identify the weakest security controls an attacker would exploit. Ammar recommends conducting similar assessments on an annual basis by internal or third-party assessors to ensure the continued success of the controls.

Contact Information

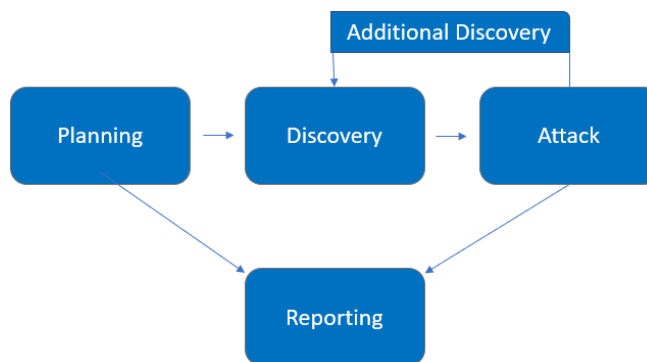
Name	Title	Contact Information
NovaFreight		
John Smith	Global Information Security Manager	Email: jsmith@democorp.com
TCM Security		
Ammar Ehab	Lead Penetration Tester	Email: ammaruchia29@tcmsec.com

Assessment Overview

From September 1st, 2026 to September 3rd, 2026, NovaFreight engaged Ammar Ehab to evaluate the security posture of its infrastructure compared to current industry best practices. That included external penetration testing. All testing performed is based on the NIST *SP 800-115 Technical Guide to Information Security Testing and Assessment*, *OWASP Testing Guide (v4)*, and *customized testing frameworks*.

Phases of penetration testing activities include the following:

- Planning – Customer goals are gathered and rules of engagement obtained.
- Discovery – Perform scanning and enumeration to identify potential vulnerabilities, weak areas, and exploits.
- Attack – Confirm potential vulnerabilities through exploitation and perform additional discovery upon new access.
- Reporting – Document all found vulnerabilities and exploits, failed attempts, and company strengths and weaknesses.



Assessment Components

External Penetration Test

An External penetration test emulates the role of an attacker from outside the network and the company. An engineer will scan the website to identify potential web

vulnerabilities, such as: XSS(Cross-Site Scripting), CSRF(Cross-Site-Request-Forgery), SSRF(Server-Side-Request-Forgery), SQLi(SQL injection), and more. The engineer will seek to gain root access.

Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Risk Factors

Risk is measured by two factors: Likelihood and Impact:

Likelihood

Likelihood measures the potential of a vulnerability being exploited. Ratings are given based on the difficulty of the attack, the available tools, attacker skill level, and client environment.

Impact

Impact measures the potential vulnerability’s effect on operations, including confidentiality, integrity, and availability of client systems and/or data, reputational harm, and financial loss.

Scope

Assessment	Details
External Penetration Test	NovaFreight.com

Scope Exclusions

Per client request, TCMS did not perform any of the following attacks during testing:

- Denial of Service (DoS)
- Phishing/Social Engineering

All other attacks not specified above were permitted by NovaFreight.

Executive Summary

Ammar Ehab evaluated NovaFrieght’s external security posture through penetration testing from September 1st, 2026 to September 3rd, 2026. The following sections provide a high-level overview of vulnerabilities discovered, successful and unsuccessful attempts, and strengths and weaknesses.

Scoping and Time Limitations

Scoping during the engagement did not permit denial of service or social engineering across all testing components.

Time limitations were in place for testing. Internal network penetration testing was permitted for ten (3) business days.

Testing Summary

The network assessment evaluated NovaFreight's external network security posture. From an external perspective, Ammar Ehab performed vulnerability scanning against the website to evaluate the overall security of the website. I also performed XSS, SQLi, CSRF, IDOR, Path Traversal, password spray attack and Broken authentication. Beyond vulnerability scanning and OWASP TOP 10 web attacks, I evaluated other potential risks, such as the ability of the user to change the email without getting confirmation and the database not storing password in hashes, default network's security posture. Through these attacks I was able to get user admin access and the database access to view all the users and password and was able to get support user access

Tester Notes and Recommendations

During testing, three constants stood out: a weak password policy, weak query handling for SQL and no confirmation for changing user data. The weak password policy led to the initial compromise of accounts and is usually one of the first footholds an attacker attempts to use in a network. The presence of a weak password policy is backed up by the evidence of me cracking admin account passwords.

We recommended that NovaFreight re-evaluates their current password policy and considers a policy of 15 characters or more for their regular user accounts and 30 characters or more for their Administrator accounts. We also recommend that NovaFreight explores password blacklisting and will be supplying a list of cracked user passwords for the team to evaluate. Finally, a Privilege Access Management solution should be considered.

Weak Query Handling for SQL led for the ability to login to the accounts without the needing of the password. The presence of SQLi vulnerability is backed up by the evidence of me logging to the admin account without the need of password

Ensure all database calls utilize parameterized to separate SQL executable code from user-supplied input completely. Never concatenate string inputs into raw SQL statements.

During testing, it was observed that critical user profile and configuration data could be modified without secondary verification or confirmation prompts. This increases the risk of unauthorized account takeover, Cross-Site Request Forgery (CSRF), and Insecure Direct Object Reference (IDOR) attacks.

We Recommend that NovaFreight website Require users to re-enter their current account password before saving high-risk profile changes (e.g., changing email addresses, password updates, or modifying billing detail

Remediation Roadmap: Short-, Medium-, and Long-Term Priorities

The findings above are regrouped here by recommended implementation timeframe, to help NovaFreight sequence remediation work. Short-term items are fast, high-impact fixes; medium-term items require more development effort or design work; long-term items are program-level controls that reduce the likelihood of similar findings recurring.

Short-Term (0-30 Days)

- **SQL Injection (both findings)** – convert the /login authentication query and the shipment search query to parameterized queries / prepared statements immediately; this is the single highest-impact fix in the assessment and requires no architectural change.
- **Broken Access Control (Admin Panel)** – stop trusting the client-supplied role cookie – add a server-side check on every /admin route that re-derives the user's role from the authenticated session/database record.
- **Sensitive Information Disclosure** – remove the publicly reachable backup/hidden files from web-accessible directories.
- **Cross-Site Request Forgery (CSRF)** – require re-entry of the current password before the email-update endpoint commits the change, as an immediate stopgap ahead of a full anti-CSRF token rollout.
- **Unrestricted File Upload** – add an extension/MIME-type allow-list to the upload handler as an immediate stopgap (full remediation – storage relocation, content validation – is tracked as medium-term below).

Medium-Term (1-3 Months)

- **Unrestricted File Upload** – validate uploaded file content server-side (magic-byte checks, not just extension), store uploads outside the web root with randomized, non-executable filenames, and strip execute permissions from the upload directory.
- **Broken Authentication (JWT Validation)** – replace the custom JWT implementation with an established, actively maintained library; enforce a fixed, server-side algorithm allow-list and securely manage signing

keys.

- **Insecure Direct Object Reference (IDOR)** – add authorization checks on every object reference (invoices and similar records) to verify the requested record belongs to the authenticated user's session.
- **Stored Cross-Site Scripting (XSS)** – apply context-aware output encoding to all user-generated content rendered in the admin interface and deploy a strict Content Security Policy (CSP).
- **Path Traversal** – replace the direct filename parameter on the support-desk download feature with an indirect reference (e.g. a database ID) resolved to a file path server-side.
- **Weak Credential Storage** – migrate password storage to a strong, modern algorithm (Argon2, bcrypt, or PBKDF2) with a unique per-user salt, and force a password reset for every account exposed in the disclosed credential table.
- **Server-Side Request Forgery (SSRF)** – implement a strict allow-list of permitted destinations for the admin URL-preview/label fetch tool and explicitly block requests to internal and link-local IP ranges.
- **Cross-Site Request Forgery (CSRF)** – **full remediation** – require POST/PUT/DELETE (never GET) for all state-changing actions and implement synchronizer anti-CSRF tokens or SameSite cookie attributes across the application.

Long-Term (3-6+ Months, Ongoing Program)

- **Secure SDLC practices** – require security-focused code review for all changes touching authentication, authorization, or file handling, and add security-relevant unit/regression tests for each finding above so a fix cannot silently regress.
- **Automated security testing in CI/CD** – integrate SAST and DAST scanning into the build pipeline to catch injection, access-control, and encoding defects before they reach production.
- **Centralized authorization** – consolidate role/permission checks into a single, centrally enforced authorization layer rather than per-route checks, reducing the chance that a new endpoint ships without one.
- **Credential and secrets management** – adopt a dedicated secrets-management solution for signing keys, API credentials, and database credentials, and rotate any credential that was exposed during this assessment.
- **Recurring third-party assessment** – schedule a follow-up penetration test after remediation to validate fixes, and repeat on a regular cadence (e.g. annually, or before major releases).
- **Security awareness and secure-coding training** – provide the development team with secure-coding training focused on the OWASP Top 10 categories represented in this report (injection, broken access control, and broken authentication in particular).
- **Defense in depth** – deploy a Web Application Firewall (WAF) as an additional layer – not a replacement for the root-cause fixes above.

Detailed Findings

The following findings provide detailed technical evidence gathered during testing for a subset of the vulnerabilities listed in the Vulnerability Summary & Report Card below.

Finding: SQL Injection (Authentication Bypass) (Critical)

Description:	The /login endpoint's username parameter is passed to the backend authentication query without sanitization or parameterization. Submitting a comment-based SQL injection payload in the username field, together with an arbitrary password value, authenticated the request without validating a real credential (see Figure 1).
Risk:	Likelihood: High – the technique is well documented and requires no special tooling beyond an intercepting proxy. Impact: Critical – results in full authentication bypass; testing confirmed access to the admin account without a valid password.
System:	NovaFreight web application — POST /login (http://127.0.0.1:8000/login)
Tools Used:	Burp Suite, manual testing
References:	OWASP: SQL Injection
Evidence:	

```
POST /login HTTP/1.1
Host: 127.0.0.1:8000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 37
Origin: http://127.0.0.1:8000
Connection: keep-alive
Referer: http://127.0.0.1:8000/login
Cookie: sid=ea83d550f32bb63f0580949d7fbf114c880abeb5cbf40c58; role=admin
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i

username=%27admin+--+&password=admin
```

Figure 1: POST /login request containing a SQL injection payload in the username parameter

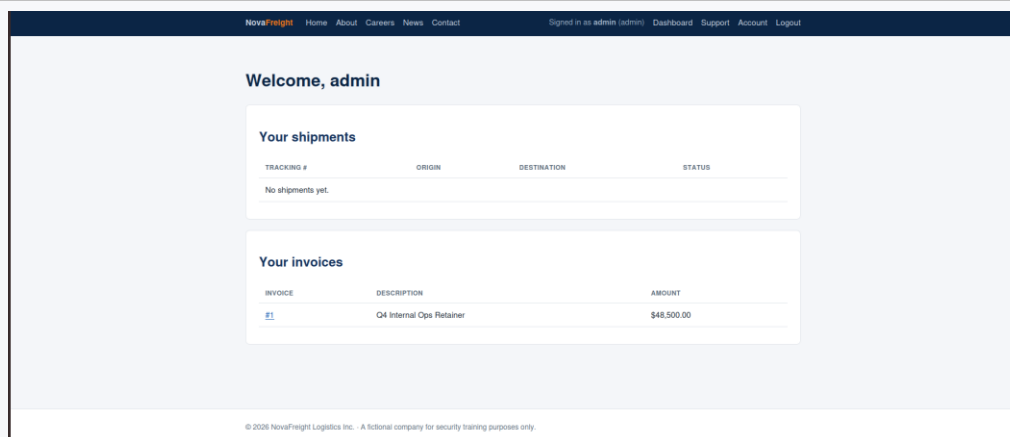


Figure 1b: Post-exploitation view of the admin account's dashboard after the SQL injection authentication bypass

Remediation

Use parameterized queries or prepared statements (or a vetted ORM) for all database calls involved in authentication; never concatenate user-supplied input into SQL statements. Apply strict server-side input validation and run the database account under least-privilege permissions as defense in depth.

Finding: SQL Injection (Data Extraction) (Critical)

Description:	The shipment search feature passes user-supplied input directly into the backend SQL query without parameterization or sanitization. A UNION-based injection payload in the search parameter allowed data to be extracted from database tables outside the normal search scope (e.g. the user credential table), independent of the login-form injection documented in the Authentication Bypass finding above.
Risk:	Likelihood: High – the technique is well documented and requires no special tooling beyond an intercepting proxy. Impact: Critical – allows extraction of arbitrary data from the underlying database, including user records outside the intended search scope.
System:	NovaFreight web application — shipment search feature
Tools Used:	Burp Suite, manual testing
References:	OWASP: SQL Injection

Evidence:**Shipment Search**

Tracking number, origin, or destination

* ORDER BY 5--

Search

Query failed.

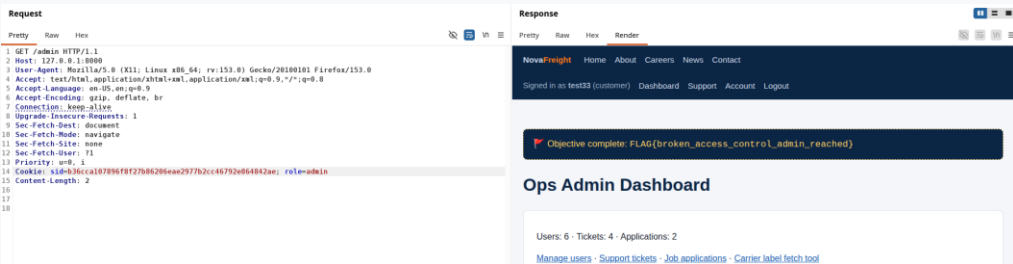
1st ORDER BY term out of range - should be between 1 and 4

SQL: SELECT tracking_number, origin, destination, status FROM shipments WHERE tracking_number LIKE '%* ORDER BY 5--%' OR o

Remediation

Enforce parameterized queries (prepared statements) on the search feature and sanitize all user-supplied input. Run the application's database account under least-privilege permissions so it cannot read tables outside its required scope.

Finding: Broken Access Control (Admin Panel) (Critical)

Description:	Authorization for the /admin panel and its sub-routes is derived from a client-controlled role cookie rather than being independently verified server-side against the authenticated session. Setting role=admin on an authenticated low-privilege session's cookie granted access to the admin dashboard and to /admin/users, which returned the full user table — including usernames, emails, and password hashes — for every account. Resetting the same cookie to role=customer on an otherwise identical request correctly returned an “Admins only” response, confirming the cookie value, not server-side session state, determines access.
Risk:	Likelihood: High – requires only editing a cookie value in an intercepting proxy; no credential guessing is needed. Impact: Critical – full administrative access and disclosure of every user account's credentials (password hashes).
System:	NovaFreight web application — /admin, /admin/users (http://127.0.0.1:8000)
Tools Used:	Burp Suite (Repeater)
References:	OWASP Top 10:2025 – A01 Broken Access Control
Evidence:	 The screenshot displays the Burp Suite interface. On the left, the 'Request' tab shows an HTTP GET request to /admin. The raw request text includes the cookie 'sid=bhica10796f8f27b86296ee297b2cc46792a04042ae; role=admin'. On the right, the 'Response' tab shows the application's response, which includes a success message: 'Objective complete: FLAG[broken_access_control_admin_reached]' and the 'Ops Admin Dashboard' interface.
Figure 2: role=admin cookie grants access to the Ops Admin Dashboard	

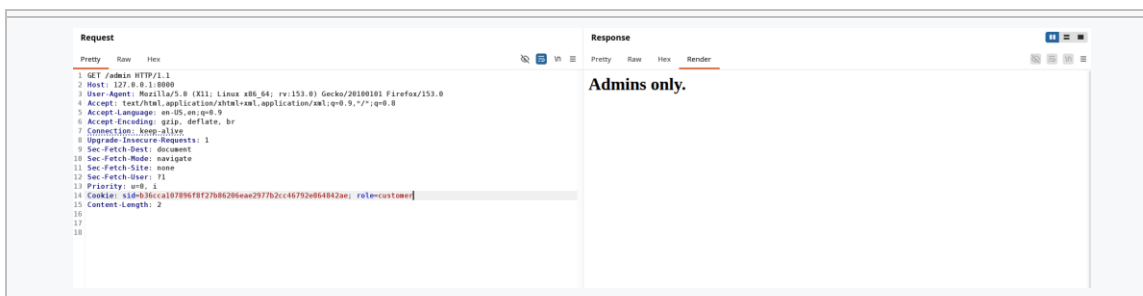


Figure 3: role=customer cookie on the same session is correctly denied (“Admins only”)

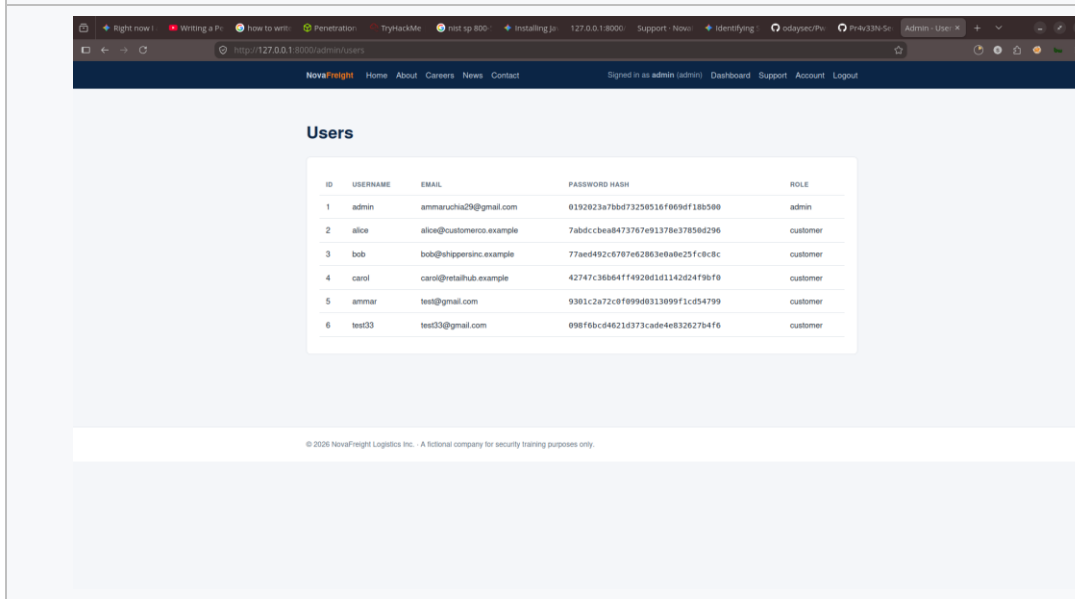


Figure 4: /admin/users discloses all user accounts and password hashes once admin access is obtained

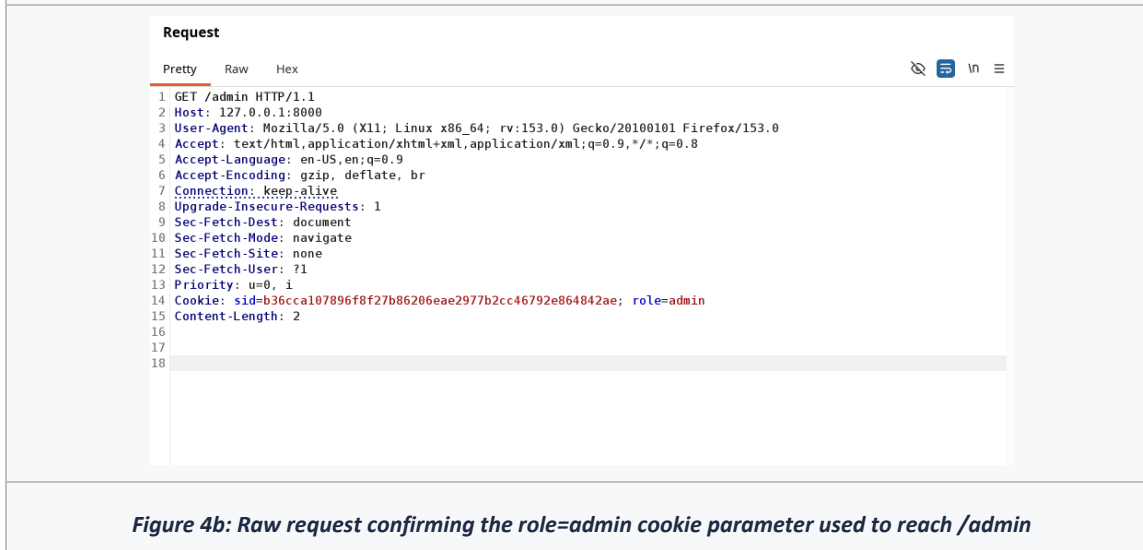


Figure 4b: Raw request confirming the role=admin cookie parameter used to reach /admin

Remediation

Enforce authorization server-side against the authenticated user's session/database record on every /admin route; never trust a client-supplied cookie, header, or hidden field to determine role or privilege. Re-validate role on each request and re-issue session identifiers on any privilege change.

Finding: Unrestricted File Upload (Critical)

Description:	The support ticket attachment feature accepts uploaded files without validating extension, MIME type, or content. A PHP file (cmd.php) was uploaded successfully as an attachment on Ticket #4 and stored with its original, executable extension intact. The tester subsequently located and retrieved the stored file via the admin ticket view (Figures 5b–5c), confirming the upload path was reachable outside the ticket UI. Server-side code execution of the uploaded file was not attempted or confirmed as part of this assessment; this finding is reported as unrestricted file upload, not remote code execution.
Risk:	Likelihood: High – no client- or server-side file-type restriction was encountered during testing. Impact: Critical – if the storage location is web-accessible and script execution is enabled there, an uploaded PHP file can be requested directly to achieve remote code execution.
System:	NovaFreight web application — Support ticket attachments (Ticket #4)
Tools Used:	Web browser, manual testing
References:	OWASP: Unrestricted File Upload
Evidence:	

Ticket #4: test

From: test33 - [open](#)

testststststst

Attachment: [cmd.php](#)

Reply

Send reply

Figure 5: cmd.php accepted and stored as a support ticket attachment

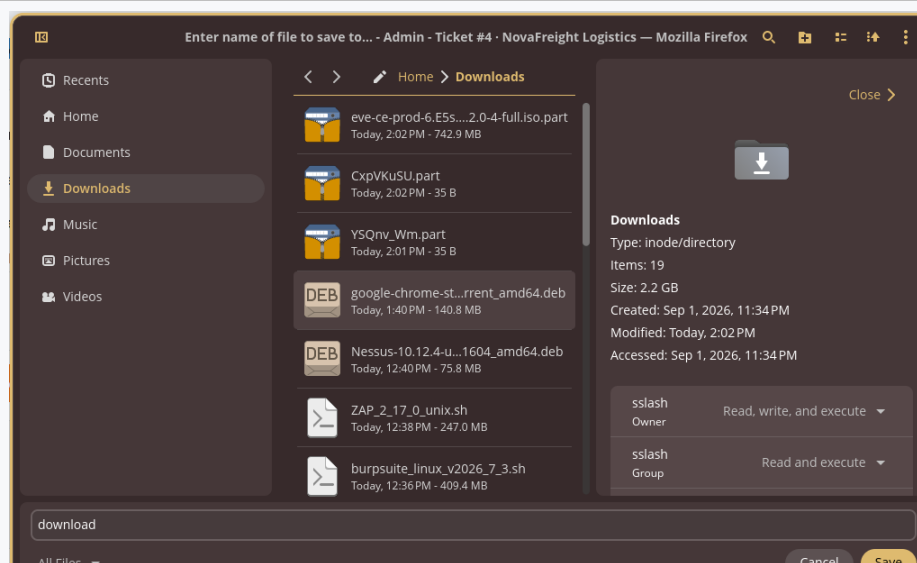
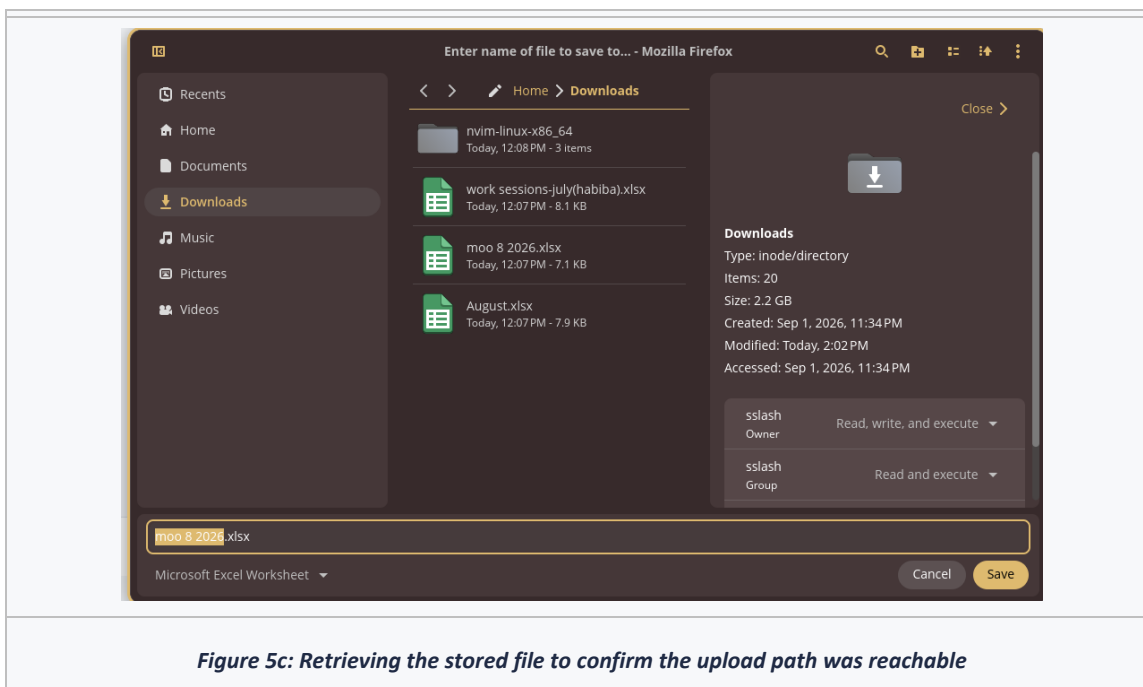


Figure 5b: Locating the stored attachment from the admin ticket view



Remediation

Validate uploaded file extensions and MIME types against an allow-list, verify actual file content server-side (e.g., magic-byte checks), store uploads outside the web root with randomized, non-executable filenames, and strip execute permissions from the upload directory.

Finding: Stored Cross-Site Scripting (XSS) (High)

Description:	Customer-submitted support ticket content is rendered in the admin interface without adequate output encoding. A stored script payload placed in a ticket field is expected to execute in the context of an administrator's browser session when the ticket is viewed, which could be used to exfiltrate admin-only data or perform actions as the administrator.
Risk:	Likelihood: Moderate – requires an administrator to open the malicious ticket. Impact: High – executes with the privileges of an authenticated admin session.

System:

NovaFreight web application — support ticket submission / admin ticket view

Tools Used:

Web browser, manual testing

References:

OWASP: Cross-Site Scripting (XSS)

Evidence:

Support Tickets (Admin)

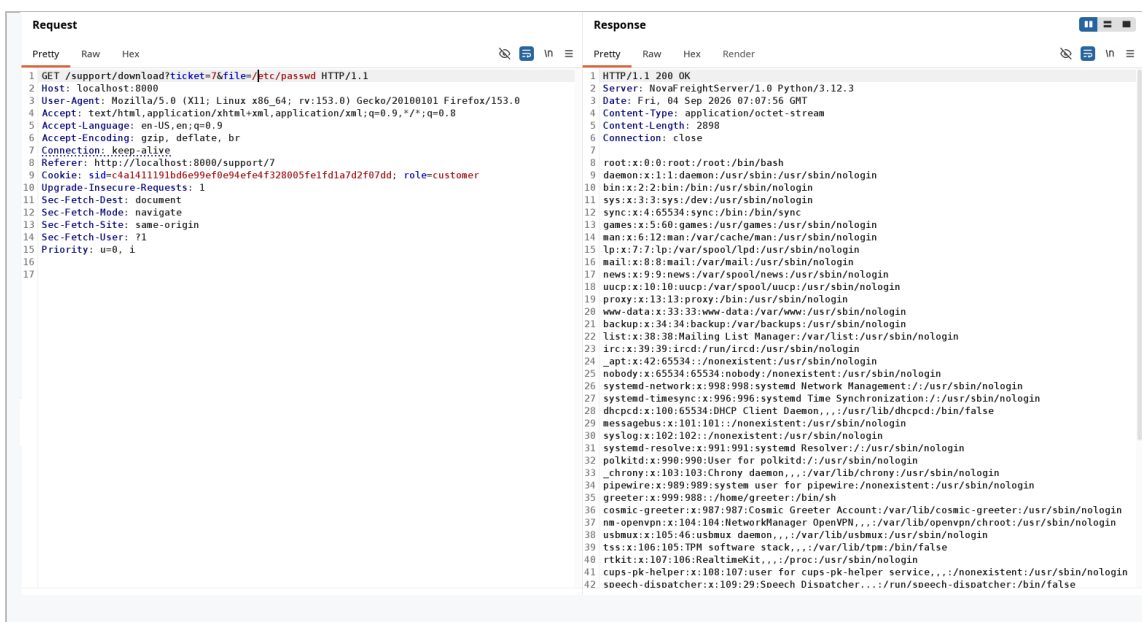
ID	CUSTOMER	SUBJECT	STATUS
#5	test33	<script>fetch("/collect?c="+encodeURIComponent(window.ADMIN_FLAG))</script>	open Open

Remediation

Apply context-aware output encoding to all user-generated content and implement a strict Content Security Policy (CSP) to reduce the impact of any encoding gaps.

Finding: Path Traversal (High)

Description:	The support desk file-download feature accepts a filename parameter that is not constrained to the intended upload directory, allowing files outside the expected scope to be requested.
Risk:	Likelihood: Moderate – requires locating the download endpoint and manipulating the filename parameter. Impact: High – can expose files never meant to be reachable over HTTP.
System:	NovaFreight web application — support desk file-download feature
Tools Used:	Web browser / Burp Suite, manual testing
References:	OWASP: Path Traversal
Evidence:	



Remediation

Reject relative path modifiers (e.g. `../`) in the filename parameter. Use indirect object references (e.g. database IDs) mapped to files server-side instead of accepting a direct filename from the client.

Finding: Weak Credential Storage (High)

Description:	Passwords recovered from the user table disclosed via the Broken Access Control finding above are stored as short, fixed-length hexadecimal values consistent with a fast, unsalted hash rather than a modern password-hashing algorithm. This makes credentials practical to recover offline once the table is obtained.
Risk:	Likelihood: High – requires only the credential dump already obtained above, plus offline cracking; no additional access is needed. Impact: High – recovered plaintext passwords can be reused against other accounts or systems if credentials are shared.
System:	NovaFreight web application — user credential storage (disclosed via <code>/admin/users</code>)
Tools Used:	Disclosed password hash table; offline cracking
References:	OWASP: Password Storage Cheat Sheet
Evidence:	

Users				
ID	USERNAME	EMAIL	PASSWORD HASH	ROLE
1	admin	ammaruchia29@gmail.com	0192023a7bbd73250516f069df18b500	admin
2	alice	alice@customerco.example	7abdcbea8473767e91378e37850d296	customer
3	bob	bob@shippersinc.example	77aed492c6707e62863e0a0e25fc0c8c	customer
4	carol	carol@retailhub.example	42747c36b64ff4920d1d1142d24f9bf0	customer
5	ammar	test@gmail.com	9301c2a72c0f099d0313099f1cd54799	customer
6	test33	test33@gmail.com	098f6bcd4621d373cade4e832627b4f6	customer

Figure: Disclosed password table — short, fixed-length hash values consistent with a fast, unsalted hashing algorithm

Remediation

Migrate password storage to a strong, modern cryptographic algorithm (e.g. Argon2, bcrypt, or PBKDF2) with a unique, random salt per user.

Finding: Cross-Site Request Forgery (CSRF) (High)

Description:	The Account Settings “Update email” function changes a sensitive account attribute without requiring the current password or any secondary confirmation. This was demonstrated by changing the admin account's email address from ops-admin@novafreight.example to an external address directly through the form, with no re-authentication prompt at any point.
Risk:	Likelihood: Moderate – exploitation depends on whether the endpoint also lacks a valid anti-CSRF token, which was not independently confirmed on the raw request. Impact: High – if combined with a missing anti-CSRF token, this action could be triggered by a malicious cross-site request against an authenticated victim. Independently of CSRF, the absence of re-authentication also weakens protection against session hijacking, since a stolen session alone is enough to redirect account notifications to an attacker-controlled address.
System:	NovaFreight web application — Account Settings, email update function

Tools Used:	Web browser, manual testing
References:	OWASP: Cross-Site Request Forgery (CSRF)

Evidence:

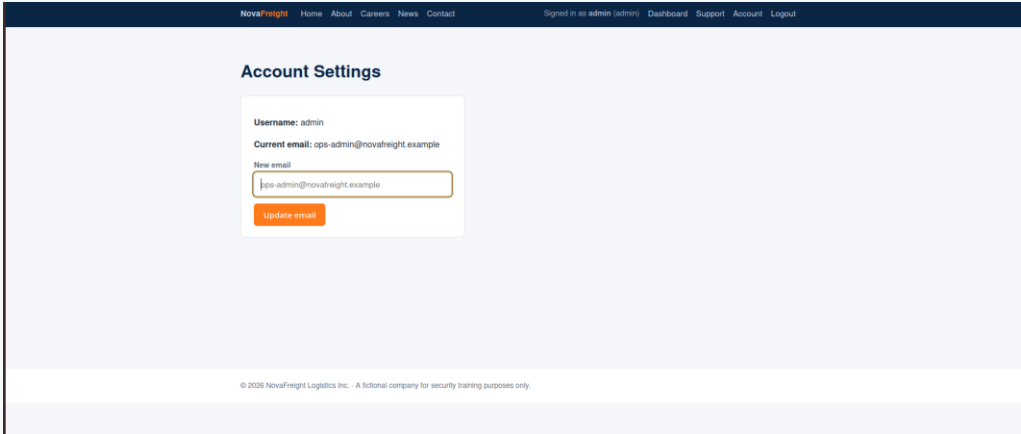


Figure 6: Admin account's email prior to the change (ops-admin@novafreight.example)

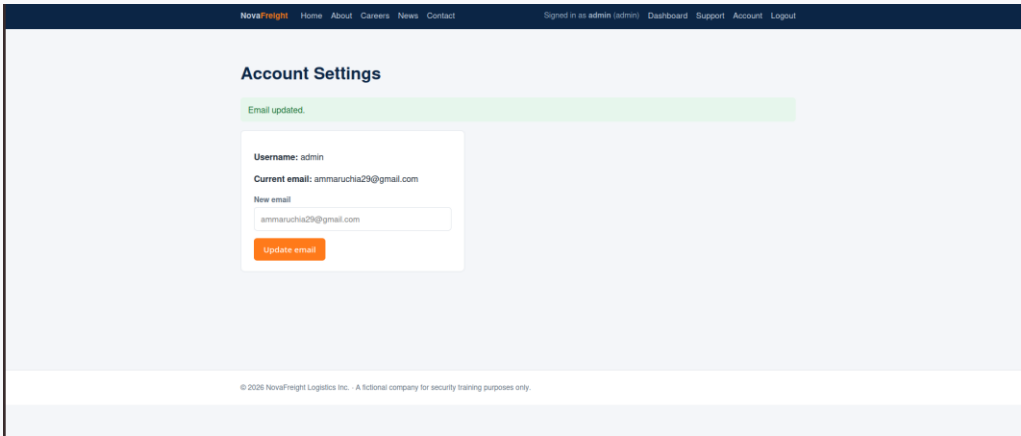


Figure 7: Confirmation that the email was updated, with no password or secondary confirmation required

Remediation

Require re-entry of the current password (or another step-up authentication factor) before committing sensitive account changes such as email or password updates. Confirm the endpoint only accepts state-changing requests via POST/PUT with a valid, per-session anti-CSRF token.

Vulnerability Summary & Report Card

The following tables illustrate the vulnerabilities found by impact and recommended remediations:

Internal Penetration Test Findings

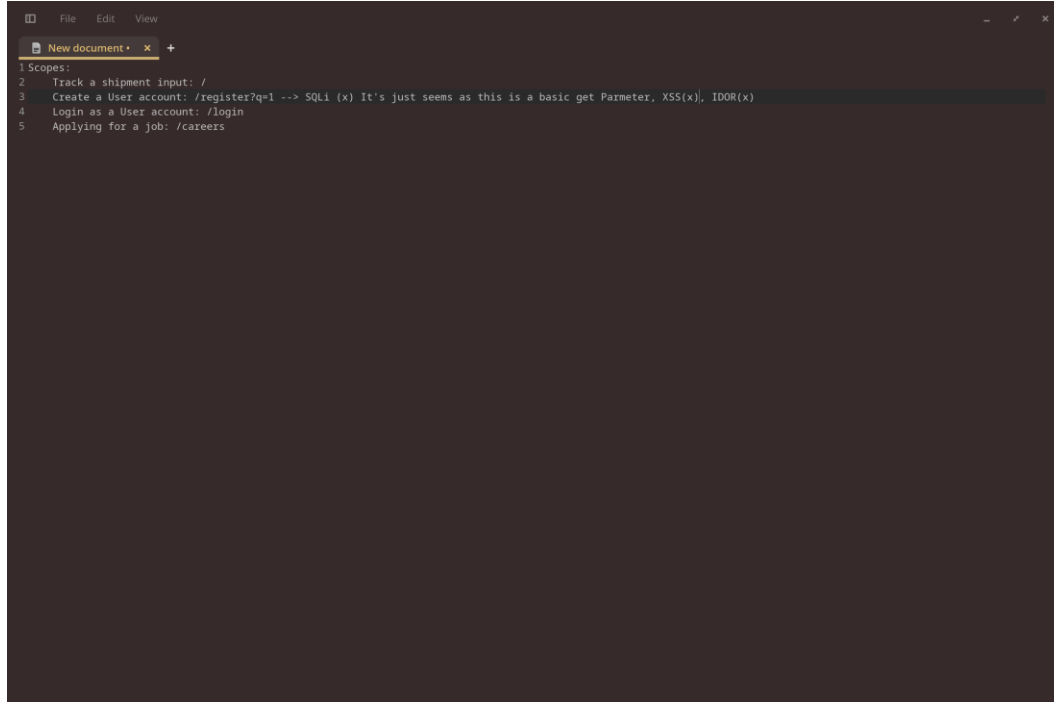
5	6	1	0	0
Critical	High	Moderate	Low	Informational

Finding	Severity	Recommendation
External Penetration Test		
SQL Injection (Authentication Bypass) Admin login mechanism is vulnerable to SQLi, allowing unauthorized access.	Critical	Implement parameterized queries (Prepared Statements) or use a secure ORM for all database interactions.
SQL Injection (Data Extraction) The shipment search feature allows UNION-based data exfiltration.	Critical	Enforce parameterized queries on the search feature and sanitize all user-supplied input.
Broken Access Control (Admin Panel) Administrative endpoints lack proper authorization checks.	Critical	Implement strict server-side Role-Based Access Control (RBAC) on all <code>/admin</code> routes.
Unrestricted File Upload Careers and support ticket forms lack type/content validation, permitting malicious file uploads.	Critical	Validate file extensions/MIME types, store uploads outside the web root, and strip execute permissions from upload directories.
Broken Authentication (JWT Validation) Custom JWT implementation improperly trusts client-supplied parameters, allowing token forgery.	Critical	Use established JWT libraries, enforce strict algorithm verification and securely manage signing keys.

Insecure Direct Object Reference (IDOR) Low-privilege users can view internal/other users' invoices by manipulating the ID parameter.	High	Implement authorization checks on object references to verify the requested record belongs to the authenticated user's session.
Stored Cross-Site Scripting (XSS) Support ticket payloads execute in the admin's browser session.	High	Apply context-aware output encoding for all user-generated content and implement a strict Content Security Policy (CSP).
Path Traversal Support desk file-download feature fails to constrain the filename parameter.	High	Reject relative path modifiers (e.g., <code>..</code> /). Use indirect object references (e.g., database IDs) mapped to files instead of direct filenames.
Sensitive Information Disclosure Publicly reachable backup/hidden files contain secrets that enable further exploitation	High	Remove sensitive backup files from web-accessible directories and configure proper directory access controls.
Server-Side Request Forgery (SSRF) Admin URL preview tool can be leveraged to access internal-only API endpoints.	High	implement a strict whitelist of permitted domains for the preview feature. Block routing to internal IPs
Weak Credential Storage Database credentials are inadequately hashed, allowing offline plaintext recovery.	High	Migrate password storage to a strong, modern cryptographic algorithm (e.g., Argon2, bcrypt, PBKDF2) with unique per-user salts.
Cross-Site Request Forgery (CSRF) State-changing account settings accept GET requests and lack anti-CSRF tokens.	High	Require POST/PUT/DELETE for all state-changing actions and implement synchronizer token patterns (anti-CSRF tokens) or <code>Samesite</code> cookie attributes.

Appendix A: Tester Scope Notes

The following is a reproduction of the tester's working notes on application scope and initial route mapping, captured during the discovery phase of the engagement.

A screenshot of a text editor window with a dark background. The window has a title bar with 'File', 'Edit', and 'View' menus. Below the title bar is a tab labeled 'New document'. The text content is as follows:

```
1 Scopes:  
2   Track a shipment input: /  
3   Create a User account: /register?q=1 --> SQLi (x) It's just seems as this is a basic get Parmeter, XSS(x)], IDOR(x)  
4   Login as a User account: /login  
5   Applying for a job: /careers
```